

Лабораторна робота №3

Тема: OpenID Connect

Мета роботи: навчитися підключати OpenID Connect.

Практичні завдання

Завдання 1: Додайте таблицю з користувачами

Для реалізації системи ідентифікації користувачів за протоколом OpenID Connect було розпочато роботу з розширення архітектури бази даних. У файлі schema.prisma (рис. 1) було спроектовано модель User, де ключовим унікальним ідентифікатором визначено поле email, що є необхідною умовою для інтеграції зі сторонніми провайдерами (Google, GitHub). Також було встановлено реляційний зв'язок "один-до-багатьох" між таблицями користувачів та статей, що дозволяє відстежувати авторство контенту. Після оновлення схеми було успішно застосовано міграцію до бази даних у Docker-контейнері (рис. 2), а коректність створення структури верифіковано через графічний інтерфейс Prisma Studio (рис. 3).

```
prisma > schema.prisma
1  generator client {
2    provider = "prisma-client-js"
3  }
4
5  datasource db {
6    provider = "postgresql"
7    url       = env("DATABASE_URL")
8  }
9
10 model User {
11   id        Int      @id @default(autoincrement())
12   email     String    @unique
13   name      String?
14   image     String?
15   createdAt DateTime @default(now())
16   updatedAt DateTime @updatedAt
17   articles  Article[]
18 }
19
20 model Article {
21   id        Int      @id @default(autoincrement())
22   title     String
23   body      String
24   category  String
25   createdAt DateTime @default(now())
26   authorId  Int?
27   author    User?    @relation(fields: [authorId], references: [id])
28 }
```

Рисунок 1 – Оновлена схема даних із моделлю User та зв'язками.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 – Лр.3							
Змн.	Арк.	№ докум.	Підпис	Дата								
Розроб.		Маковський М.С			Звіт з лабораторної роботи №3				Лім.	Арк.	Акрушіє	
Перевір.		Фант М.О									1	7
Реценз.									ФІКТ, гр. ВТ-23-2			
Н. Контр.												
Зав.каф.												

```

Environment variables loaded from .env
Prisma schema loaded from prisma/schema.prisma
Datasource "db": PostgreSQL database "fullstack_dev", schema "public" at "localhost:5434"

Applying migration `20260401105838_add_user_model`

The following migration(s) have been created and applied from new schema changes:

prisma/migrations/
├─ 20260401105838_add_user_model/
└─ migration.sql

Your database is now in sync with your schema.

✓ Generated Prisma Client (v6.19.2) to ./node_modules/@prisma/client in 42ms

```

Рисунок 2 – Результат виконання команди prisma migrate у терміналі.

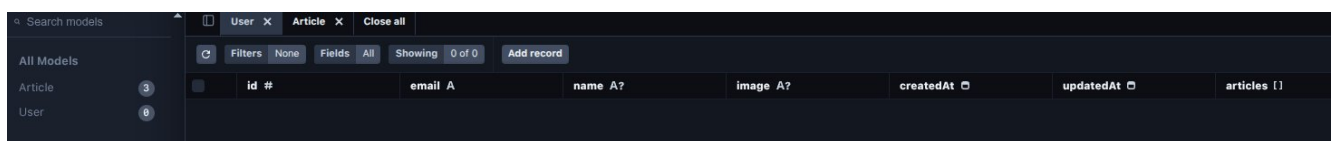


Рисунок 3 – Відображення нової таблиці User у Prisma Studio.

Завдання 2: Додайте аутентифікацію через логін/пароль

Для реалізації механізму автентифікації було інтегровано бібліотеку NextAuth.js, яка забезпечує гнучке управління сесіями та підтримку різних провайдерів ідентифікації. У проєкті було налаштовано CredentialsProvider (рис. 4), який дозволяє виконувати вхід за паролем, взаємодіючи з базою даних PostgreSQL через Prisma. Для збереження цілісності дизайну проєкту Taxi-Town було розроблено кастомну сторінку входу (рис. 5), що використовує фірмову палітру та елементи бруталістичного стилю. Використання стратегії JWT (JSON Web Token) дозволило забезпечити безпечне збереження стану автентифікації на стороні клієнта без надмірного навантаження на сервер.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.3	Арк.
						2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

src > app > api > auth > [...nextauth] > routes > handler > pages
1  import NextAuth from "next-auth";
2  import CredentialsProvider from "next-auth/providers/credentials";
3  import { PrismaAdapter } from "@auth/prisma-adapter";
4  import { PrismaClient } from "@prisma/client";
5  import bcrypt from "bcryptjs";
6
7  const prisma = new PrismaClient();
8
9  const handler = NextAuth({
10   adapter: PrismaAdapter(prisma),
11   providers: [
12     CredentialsProvider({
13       name: "Taxi-Pass",
14       credentials: {
15         email: { label: "Email", type: "email", placeholder: "driver@taxi.ua" },
16         password: { label: "Пароль", type: "password" }
17       },
18       async authorize(credentials) {
19         if (!credentials?.email || !credentials?.password) return null;
20
21         const user = await prisma.user.findUnique({
22           where: { email: credentials.email }
23         });
24
25         if (user) {
26           return user as any;
27         }
28         return null;
29       }
30     })
31   ],
32   session: { strategy: "jwt" },
33   pages: {
34     signIn: "/auth/signin",
35   },
36 });
37
38 export { handler as GET, handler as POST };

```

Рисунок 4 – Конфігурація NextAuth з використанням PrismaAdapter.

Рисунок 5 – Інтерфейс кастомної сторінки логіну.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.3	Арк.
						3
Змн.	Арк.	№ докум.	Підпис	Дата		

Завдання 3: Додайте аутентифікацію через Google

Для реалізації автентифікації користувачів за протоколом OpenID Connect було інтегровано провайдера Google. На боці Google Cloud Console було створено ідентифікатор клієнта та налаштовано URI перенаправлення для коректної роботи callback-функцій. У проекті на базі NextAuth було підключено GoogleProvider та налаштовано PrismaAdapter, що дозволяє системі автоматично реєструвати нових користувачів у базі даних PostgreSQL при першому вході. На сторінці авторизації реалізовано клієнтський функціонал ініціації сесії, що забезпечує безпечний обмін токенами між сервісом ідентифікації та додатком.

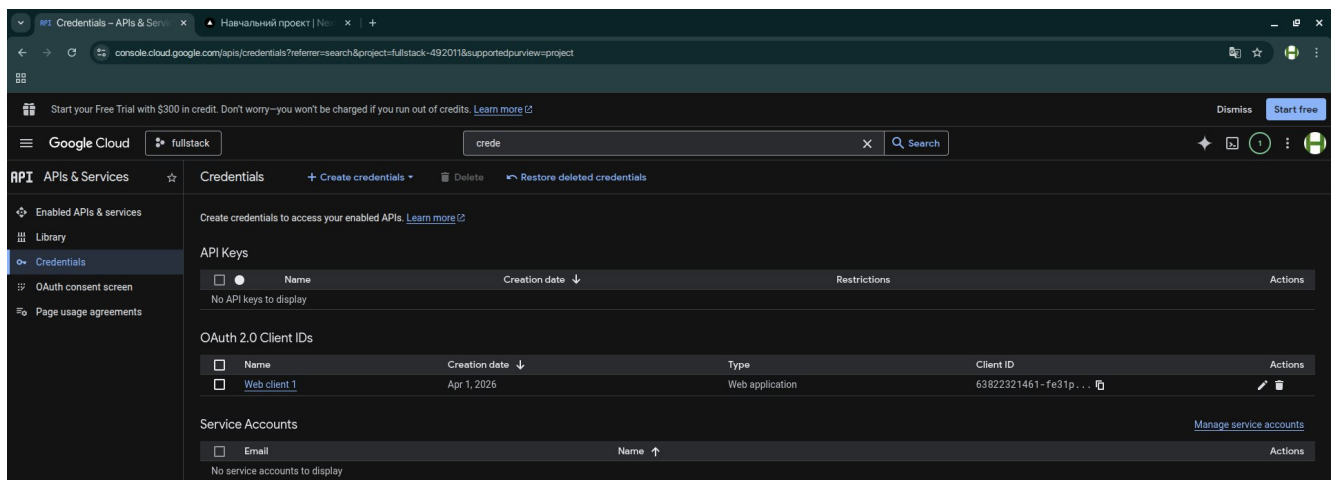


Рисунок 6 – Налаштування OAuth 2.0 Client ID у Google Cloud Console.

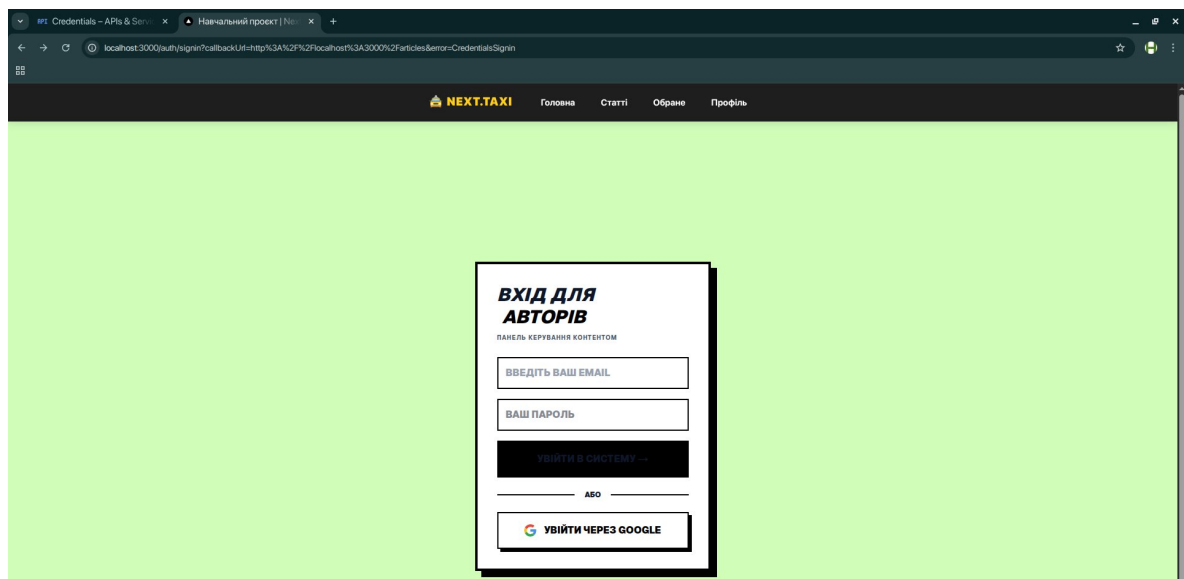


Рисунок 7 – Реалізована сторінка входу з підтримкою зовнішнього провайдера.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.3	Арк.
Змн.	Арк.	№ докум.	Підпис	Дата		4

Завдання 4: Додайте аутентифікацію через GitHub

На заключному етапі було реалізовано підтримку автентифікації через провайдера GitHub. Процес включав реєстрацію OAuth-додатка в налаштуваннях розробника GitHub (рис. 8), де було отримано унікальні ідентифікатори клієнта та налаштовано зворотні виклики (callback URL) для локального середовища розробки. У програмній частині було розширено масив провайдерів NextAuth та адаптовано клієнтську сторінку входу. Завдяки використанню PrismaAdapter, дані профілю GitHub (email, ім'я, зображення) автоматично синхронізуються з базою даних PostgreSQL при першій успішній спробі входу. Це завершує формування системи багатоканальної авторизації на платформі (рис. 9).

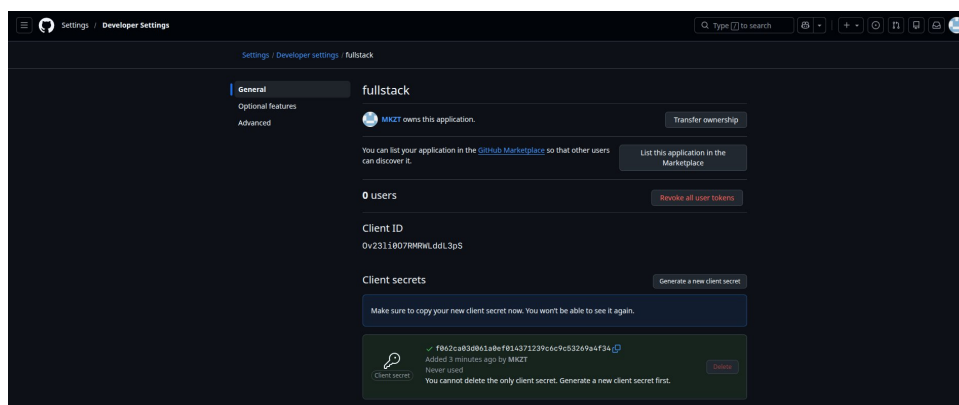


Рисунок 8 – Налаштування OAuth App у панелі керування GitHub.

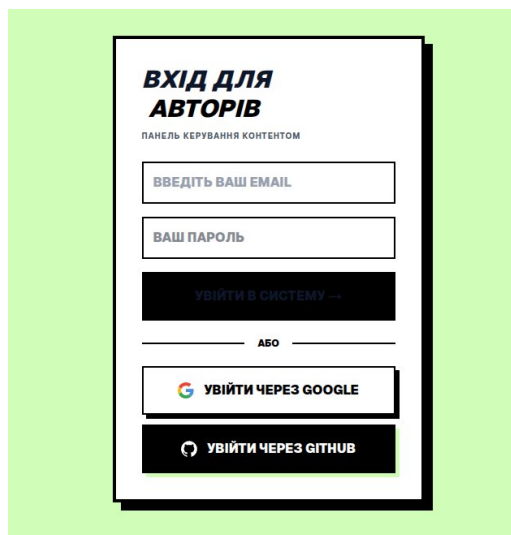


Рисунок 9 – Фінальний вигляд сторінки авторизації з підтримкою Credentials, Google та GitHub.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.3	Арк.
						5
Змн.	Арк.	№ докум.	Підпис	Дата		

Завдання 5: Додайте сторінку з профілем користувача

На заключному етапі розробки було реалізовано функціонал сторінки профілю /profile (рис. 10), яка забезпечує взаємодію користувача з його персональними даними. У програмній частині було розроблено Full-stack механізм оновлення інформації: клієнтська сторона відправляє PUT-запит на спеціально створений API-маршрут /api/user/update, де за допомогою Prisma ORM виконується модифікація запису в таблиці User бази даних PostgreSQL (Docker).

Особливу увагу приділено синхронізації стану сесії: після успішного запису в БД використовується метод update() бібліотеки NextAuth, що дозволяє миттєво відобразити нове ім'я користувача в навігаційному меню без перезавантаження сторінки (рис. 11). Для забезпечення безпеки було створено окремий розділ /profile/security, що логічно відокремлює редагування публічних даних від конфіденційних налаштувань пароля. Дизайн інтерфейсу витримано в загальному бруталістичному стилі проєкту з використанням жорстких тіней та акцентного кольору Taxi Yellow.

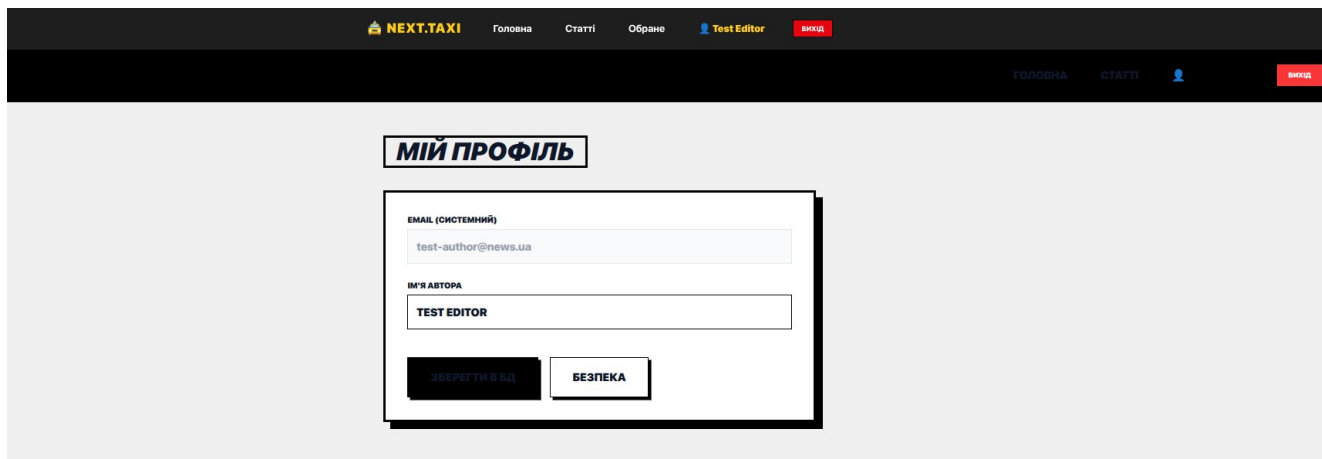


Рисунок 10 – Інтерфейс сторінки профілю з функцією глобального оновлення імені автора.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.3	Арк.
						6
Змн.	Арк.	№ докум.	Підпис	Дата		

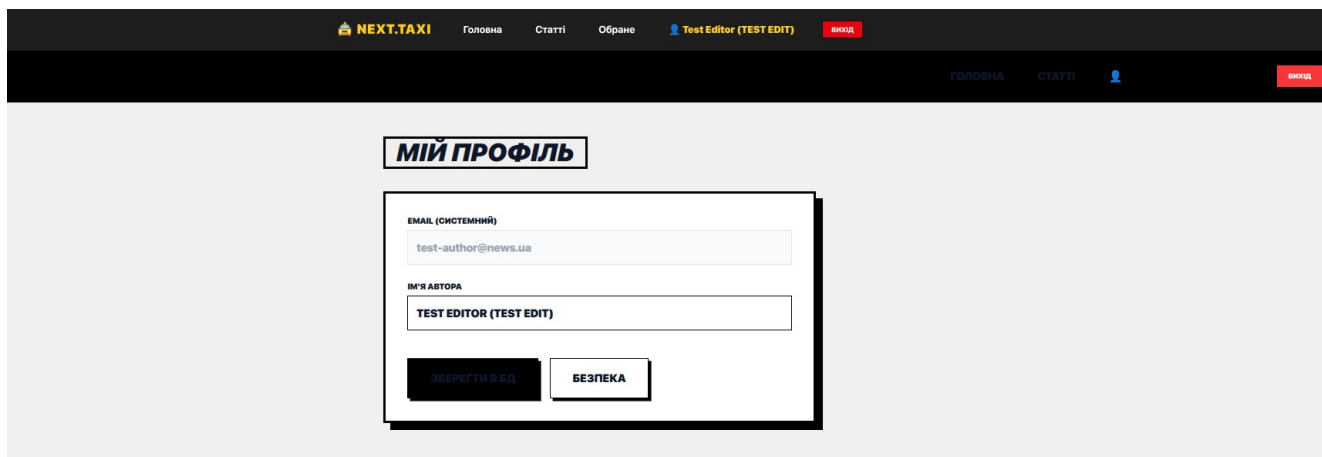


Рисунок 11 – Динамічне відображення імені залогіненого користувача в головному меню навігації.

Висновок: Під час виконання лабораторної роботи №3 було опановано практичні навички впровадження протоколу OpenID Connect та багатоканальної автентифікації у сучасний вебдодаток на базі Next.js. Завдяки використанню бібліотеки NextAuth.js та Prisma ORM, було успішно реалізовано систему авторизації через локальні облікові записи (Credentials), а також інтегровано зовнішніх провайдерів Google та GitHub через OAuth 2.0. У результаті роботи було створено повноцінний механізм керування сесіями користувачів, розроблено захищений інтерфейс профілю з можливістю глобального оновлення даних у базі PostgreSQL (Docker) та налаштовано динамічне відображення стану автентифікації в навігаційному меню додатка.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.121.8.000 –Лр.3	Арк.
						7
Змн.	Арк.	№ докум.	Підпис	Дата		